

EXPERIMENT 1

IMPLEMENTATION OF CAESAR CIPHER (PYTHON):

AIM

To implement the **Caesar Cipher** substitution technique using Python.

ALGORITHM

1. Read the plaintext from the user.
2. Read the key (shift value) from the user.
3. For each character in the plaintext:
 - If it is an uppercase letter, shift it by the key within A-Z.
 - If it is a lowercase letter, shift it by the key within a-z.
 - Leave non-alphabetic characters unchanged.
4. Display the encrypted (cipher) text.
5. Decrypt the cipher text by shifting each character backward using the same key.
6. Display the decrypted text.

Caesar Cipher in Python

CODE:

```
def encrypt(text, key):  
    cipher = ""  
    for ch in text:  
        if ch.isupper():  
            cipher += chr((ord(ch) - ord('A') + key) % 26 + ord('A'))  
        elif ch.islower():  
            cipher += chr((ord(ch) - ord('a') + key) % 26 + ord('a'))  
        else:  
            cipher += ch  
    return cipher  
  
def decrypt(cipher, key):  
    text = ""  
    for ch in cipher:
```

```

    if ch.isupper():
        text += chr((ord(ch) - ord('A') - key) % 26 + ord('A'))
    elif ch.islower():
        text += chr((ord(ch) - ord('a') - key) % 26 + ord('a'))
    else:
        text += ch

return text

plain = input("Enter the plain text: ")
key = int(input("Enter the key value: "))
cipher = encrypt(plain, key)
print("Encrypted Text:", cipher)
decrypted = decrypt(cipher, key)
print("Decrypted Text:", decrypted)

```

OUTPUT

```

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/preet/OneDrive/Desktop/csa5051/lab experiments/LAB1.py ===
Enter the plain text: HELLO
Enter the key value: 3
Encrypted Text: KHOOR
Decrypted Text: HELLO
>>>
=== RESTART: C:/Users/preet/OneDrive/Desktop/csa5051/lab experiments/LAB1.py ===
Enter the plain text: MEET ME AFTER THE PARTY
Enter the key value: 5
Encrypted Text: RJJY RJ FKYJW YMJ UFWYD
Decrypted Text: MEET ME AFTER THE PARTY
>>>

```

RESULT

Thus, the **Caesar Cipher substitution technique** was implemented successfully using Python.

EXPERIMENT 2

MONOALPHABETIC SUBSTITUTION CIPHER (PYTHON)

AIM

To implement the Monoalphabetic Substitution Cipher using Python.

ALGORITHM

1. Read the plaintext from the user.
2. Define a substitution key containing all 26 unique alphabet letters.
3. Replace each plaintext letter with its corresponding letter from the substitution key.
4. Display the encrypted (cipher) text.
5. For decryption, replace each cipher letter with its corresponding original alphabet letter.
6. Display the decrypted text.

CODE

```
import string

alphabet = string.ascii_uppercase
key = "QWERTYUIOPASDFGHJKLZXCVBNM"

plain = input("Enter the plain text: ").upper()

# Encryption
cipher = ""
for ch in plain:
    if ch.isalpha():
        index = alphabet.index(ch)
        cipher += key[index]
    else:
```

```

cipher += ch

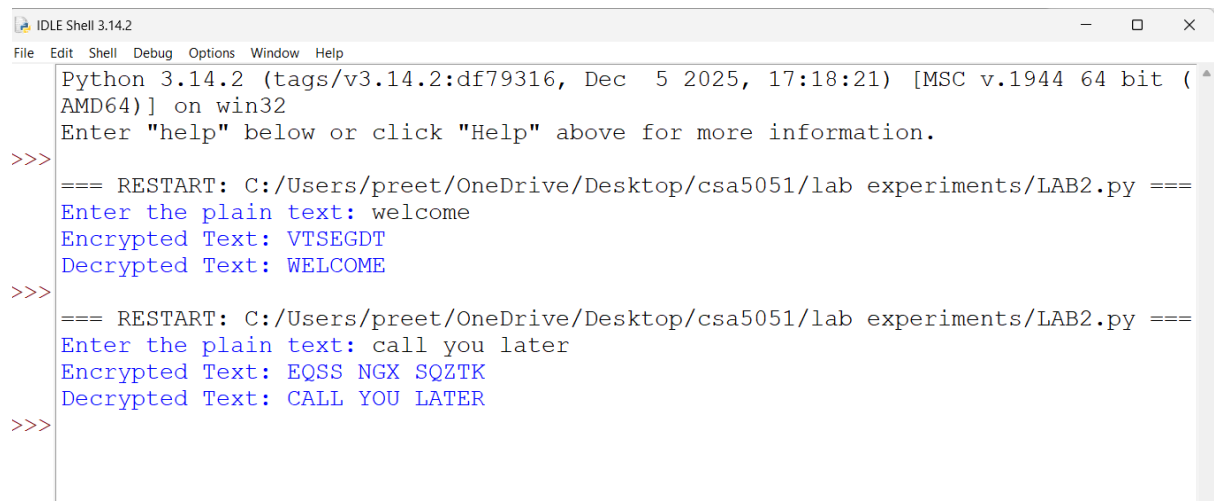
print("Encrypted Text:", cipher)

# Decryption
decrypted = ""
for ch in cipher:
    if ch.isalpha():
        index = key.index(ch)
        decrypted += alphabet[index]
    else:
        decrypted += ch

print("Decrypted Text:", decrypted)

```

OUTPUT



```

IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/preet/OneDrive/Desktop/csa5051/lab experiments/LAB2.py ===
Enter the plain text: welcome
Encrypted Text: VTSEGDT
Decrypted Text: WELCOME
>>>
=== RESTART: C:/Users/preet/OneDrive/Desktop/csa5051/lab experiments/LAB2.py ===
Enter the plain text: call you later
Encrypted Text: EQSS NGX SQZTK
Decrypted Text: CALL YOU LATER
>>>

```

RESULT

Thus, the Monoalphabetic Substitution Cipher was implemented successfully using Python.

EXPERIMENT 3

POLYALPHABETIC CIPHER (VIGENÈRE CIPHER) – PYTHON

AIM

To implement the Vigenère (Polyalphabetic) Cipher substitution technique using Python.

ALGORITHM

1. Read the plaintext from the user.
2. Read the keyword from the user.
3. Repeat the keyword until its length matches the plaintext.
4. Encrypt each character by adding the positions of the plaintext character and the corresponding key character modulo 26.
5. Display the encrypted (cipher) text.
6. Decrypt the cipher text by subtracting the key character value from the cipher character modulo 26.
7. Display the decrypted text.

PYTHON CODE

```
def generate_key(text, key):  
    key = key.upper()  
    text = text.upper()  
    new_key = ""  
    j = 0  
  
    for ch in text:  
        if ch.isalpha():  
            new_key += key[j % len(key)]  
            j += 1  
        else:  
            new_key += ch  
    return new_key
```

```
def encrypt(text, key):  
    text = text.upper()  
    key = generate_key(text, key)  
    cipher = ""  
  
    for i in range(len(text)):  
        if text[i].isalpha():  
            x = (ord(text[i]) - ord('A') + ord(key[i]) - ord('A')) % 26  
            cipher += chr(x + ord('A'))  
        else:  
            cipher += text[i]  
  
    return cipher
```

```
def decrypt(cipher, key):  
    cipher = cipher.upper()  
    key = generate_key(cipher, key)  
    plain = ""  
  
    for i in range(len(cipher)):  
        if cipher[i].isalpha():  
            x = (ord(cipher[i]) - ord(key[i])) % 26  
            plain += chr(x + ord('A'))  
        else:  
            plain += cipher[i]
```

```
return plain
```

```
plaintext = input("Enter Plain Text: ")
```

```
key = input("Enter Key: ")
```

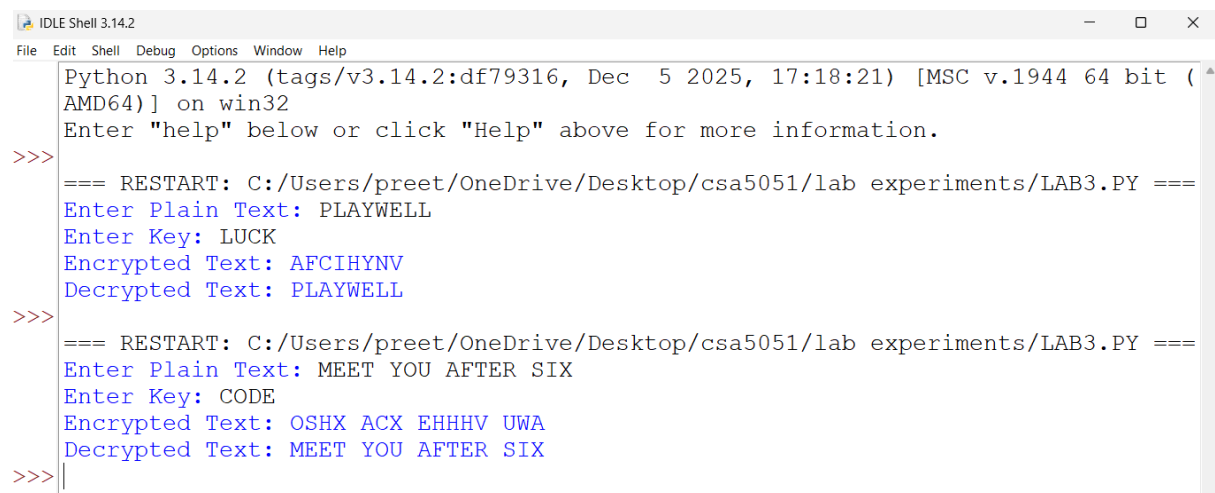
```
cipher = encrypt(plaintext, key)
```

```
print("Encrypted Text:", cipher)
```

```
plain = decrypt(cipher, key)
```

```
print("Decrypted Text:", plain)
```

OUTPUT



```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/preet/OneDrive/Desktop/csa5051/lab experiments/LAB3.PY ===
Enter Plain Text: PLAYWELL
Enter Key: LUCK
Encrypted Text: AFCIHYNV
Decrypted Text: PLAYWELL
>>>
=== RESTART: C:/Users/preet/OneDrive/Desktop/csa5051/lab experiments/LAB3.PY ===
Enter Plain Text: MEET YOU AFTER SIX
Enter Key: CODE
Encrypted Text: OSHX ACX EHHHV UWA
Decrypted Text: MEET YOU AFTER SIX
>>>
```

RESULT

Thus, the Vigenère (Polyalphabetic) Cipher substitution technique was implemented successfully using Python.

EXPERIMENT 4

HILL CIPHER

AIM

To implement the Hill Cipher substitution technique using Python.

ALGORITHM

1. Read the plaintext from the user.
2. Read the 3×3 key matrix.
3. Convert the plaintext into numerical values (A=0, B=1, ..., Z=25).
4. Multiply the plaintext vector with the key matrix modulo 26.
5. Convert the result back to characters to obtain the cipher text.
6. Display the encrypted text.

PYTHON CODE

```
key = [  
    [6, 24, 1],  
    [13, 16, 10],  
    [20, 17, 15]  
]  
  
text = input("Enter Plain Text (3 letters): ").upper()  
  
while len(text) < 3:  
    text += "X"  
  
text = text[:3]  
  
P = [ord(ch) - 65 for ch in text]  
  
C = []
```

```

for i in range(3):
    total = 0
    for j in range(3):
        total += key[i][j] * P[j]
    C.append(total % 26)

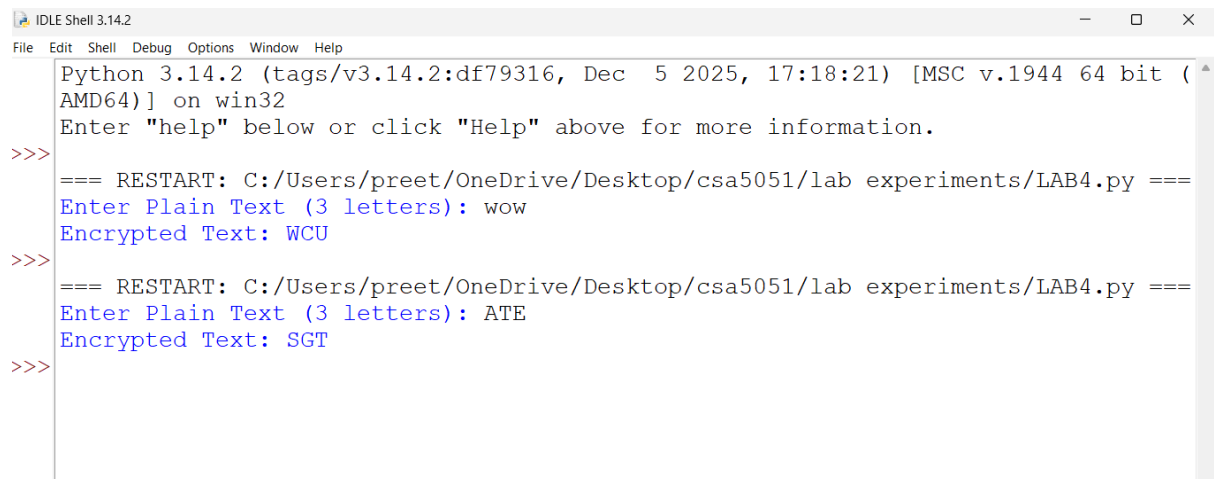
cipher = ""

for num in C:
    cipher += chr(num + 65)

print("Encrypted Text:", cipher)

```

OUTPUT



```

IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/preet/OneDrive/Desktop/csa5051/lab experiments/LAB4.py ===
Enter Plain Text (3 letters): wow
Encrypted Text: WCU
>>>
=== RESTART: C:/Users/preet/OneDrive/Desktop/csa5051/lab experiments/LAB4.py ===
Enter Plain Text (3 letters): ATE
Encrypted Text: SGT
>>>

```

RESULT

Thus the Hill Cipher substitution technique was implemented successfully using Python.

EXPERIMENT 5

PLAYFAIR CIPHER

AIM

To implement the Playfair Cipher substitution technique using Python.

ALGORITHM

1. Read the plaintext and keyword.
2. Construct the 5×5 key matrix.
3. Split the plaintext into pairs.
4. Apply Playfair encryption rules.
5. Display the encrypted text.

PYTHON CODE

```
def generate_key_table(key):  
    key = key.upper().replace("J", "I")  
    table = []  
    for c in key:  
        if c not in table and c.isalpha():  
            table.append(c)  
    for c in "ABCDEFGHIKLMNOPQRSTUVWXYZ":  
        if c not in table:  
            table.append(c)  
    return [table[i:i+5] for i in range(0, 25, 5)]  
  
def find(table, ch):  
    for i in range(5):  
        for j in range(5):  
            if table[i][j] == ch:  
                return i, j  
  
def encrypt(text, table):
```

```
text = text.upper().replace("J", "I")
text = "".join(c for c in text if c.isalpha())
```

```
pairs = []
i = 0
while i < len(text):
    a = text[i]
    if i + 1 < len(text):
        b = text[i + 1]
        if a == b:
            pairs.append((a, "X"))
            i += 1
        else:
            pairs.append((a, b))
            i += 2
    else:
        pairs.append((a, "X"))
        i += 1
```

```
cipher = ""
```

```
for a, b in pairs:
    r1, c1 = find(table, a)
    r2, c2 = find(table, b)

    if r1 == r2:
        cipher += table[r1][(c1 + 1) % 5]
        cipher += table[r2][(c2 + 1) % 5]
    elif c1 == c2:
```

```

        cipher += table[(r1 + 1) % 5][c1]

        cipher += table[(r2 + 1) % 5][c2]

    else:

        cipher += table[r1][c2]

        cipher += table[r2][c1]

return cipher

key = input("Enter Key: ")

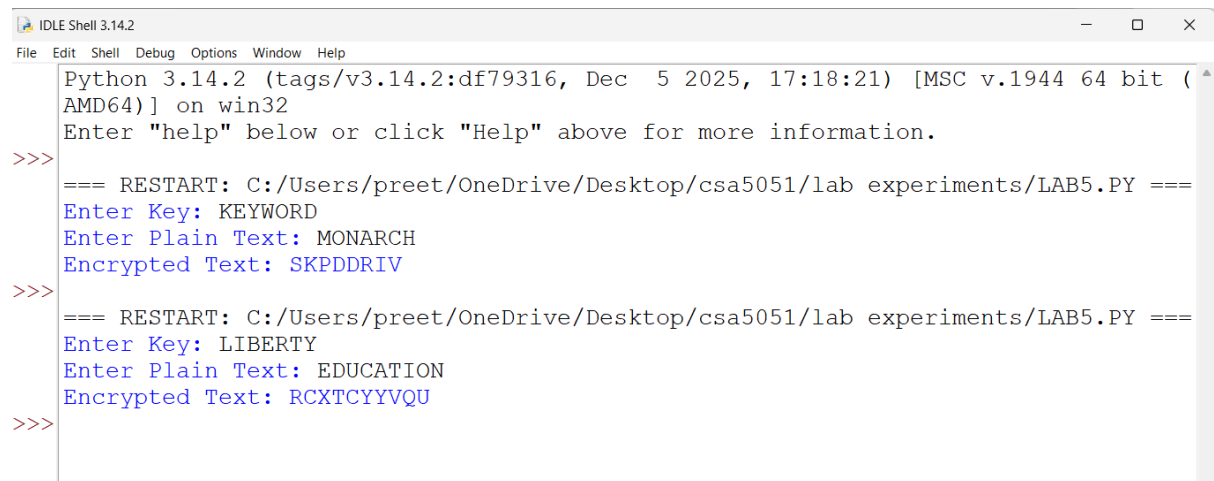
plain = input("Enter Plain Text: ")

table = generate_key_table(key)

print("Encrypted Text:", encrypt(plain, table))

```

OUTPUT



```

IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/preet/OneDrive/Desktop/csa5051/lab experiments/LAB5.PY ===
Enter Key: KEYWORD
Enter Plain Text: MONARCH
Encrypted Text: SKPDDRIV
>>>
=== RESTART: C:/Users/preet/OneDrive/Desktop/csa5051/lab experiments/LAB5.PY ===
Enter Key: LIBERTY
Enter Plain Text: EDUCATION
Encrypted Text: RCXTCYYVQU
>>>

```

RESULT

Thus the Playfair Cipher substitution technique was implemented successfully using Python.

EXPERIMENT 6

RAIL FENCE CIPHER

AIM

To implement the Rail Fence transposition technique using Python.

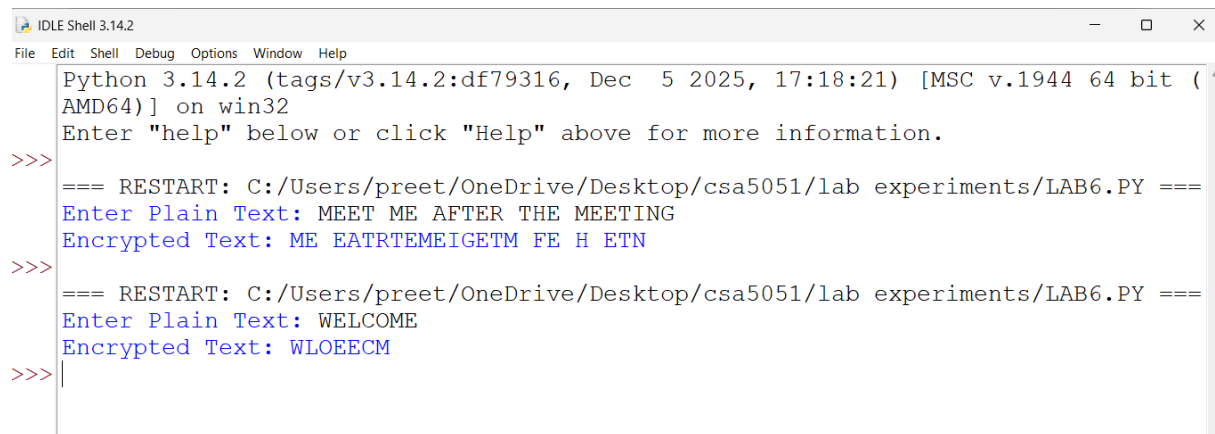
ALGORITHM

1. Read the plaintext.
2. Arrange the characters in zig-zag order.
3. Read the rows sequentially to obtain the cipher text.
4. Display the encrypted text.

PYTHON CODE

```
def encrypt(text):  
    rail1 = ""  
    rail2 = ""  
  
    for i in range(len(text)):  
        if i % 2 == 0:  
            rail1 += text[i]  
        else:  
            rail2 += text[i]  
  
    return rail1 + rail2  
  
plain = input("Enter Plain Text: ")  
  
cipher = encrypt(plain)  
  
print("Encrypted Text:", cipher)
```

OUTPUT



```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/preet/OneDrive/Desktop/csa5051/lab experiments/LAB6.PY ===
Enter Plain Text: MEET ME AFTER THE MEETING
Encrypted Text: ME EATRTEMEIGETM FE H ETN
>>>
=== RESTART: C:/Users/preet/OneDrive/Desktop/csa5051/lab experiments/LAB6.PY ===
Enter Plain Text: WELCOME
Encrypted Text: WLOEECM
>>>
```

RESULT

Thus the Rail Fence transposition technique was implemented successfully using Python.

EXPERIMENT 7

COLUMNAR TRANSPOSITION CIPHER

AIM

To implement the Columnar Transposition Cipher using Python.

ALGORITHM

1. Read the plaintext and keyword.
2. Arrange the plaintext row-wise in a matrix.
3. Number the columns according to the alphabetical order of the keyword.
4. Read the columns in sorted-key order.
5. Display the encrypted text.

PYTHON CODE

```
import math

text = input("Enter Plain Text: ").replace(" ", "").upper()
key = input("Enter Key: ").upper()

col = len(key)
row = math.ceil(len(text) / col)

fill = row * col - len(text)
text += "X" * fill

matrix = []

k = 0
for i in range(row):
    matrix.append(list(text[k:k + col]))
    k += col
```

```
order = sorted(list(enumerate(key)), key=lambda x: x[1])
```

```
cipher = ""
```

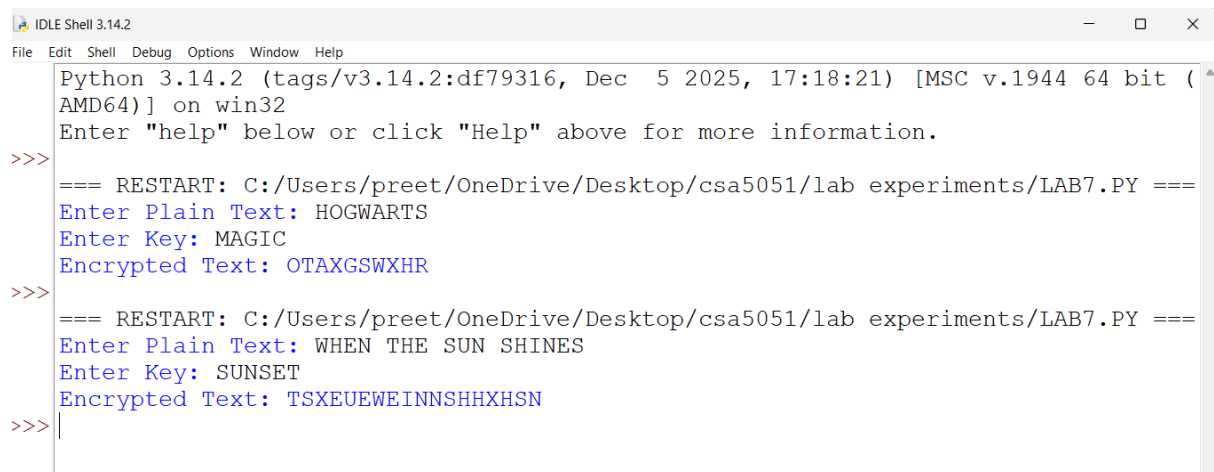
```
for index, _ in order:
```

```
    for r in matrix:
```

```
        cipher += r[index]
```

```
print("Encrypted Text:", cipher)
```

OUTPUT



```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/preet/OneDrive/Desktop/csa5051/lab experiments/LAB7.PY ===
Enter Plain Text: HOGWARTS
Enter Key: MAGIC
Encrypted Text: OTAXGSWXHR
>>>
=== RESTART: C:/Users/preet/OneDrive/Desktop/csa5051/lab experiments/LAB7.PY ===
Enter Plain Text: WHEN THE SUN SHINES
Enter Key: SUNSET
Encrypted Text: TSXEUEWEINNSHHXHSN
>>>
```

RESULT

Thus the Columnar Transposition Cipher was implemented successfully using Python.